Queues

DSA27 — Lecture 07 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 7 · Fall 2026

Where We Are

One end was easy. Now we need both.

Week 6: the **stack** touches **one** end. Put the top at the cheap end, done.

Today: the **queue** adds at one end and removes at the **other**.

- A dynamic array is cheap at the **end** only.
- A singly linked list is cheap at the **head** (and, with a tail, for adding at the end).

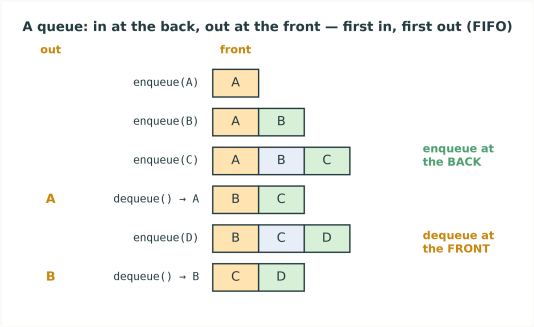
So which end goes where, and what happens to the expensive end?

Today

1. The Queue ADT: enqueue, dequeue — FIFO
2. The obvious design, `SlowQueue`, and its $O(n)$ dequeue
3. The ring buffer: `CircularQueue`, every operation $O(1)$
4. Full or empty? — and growing a ring without scrambling it
5. The linked queue, and a queue made of two stacks
6. Measured: the gap is the lesson
7. Where queues are used — and the deque

The Queue ADT

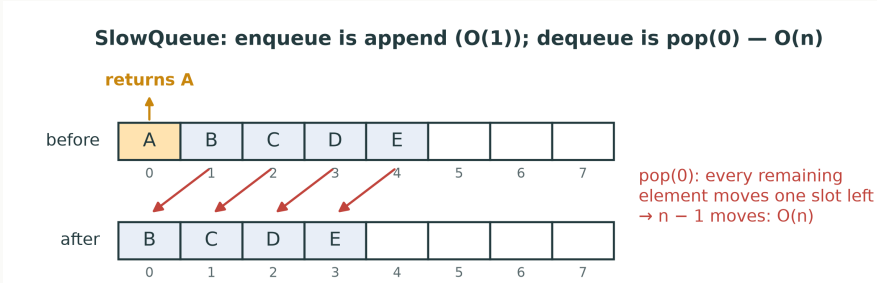
First in, first out



Operation	Meaning	Cost
`enqueue(x)`	add x at the **back**	$O(1)$
`dequeue()`	remove and return the **front** (`IndexError` if empty)	$O(1)$
`is_empty()`, `len(q)`		$O(1)$

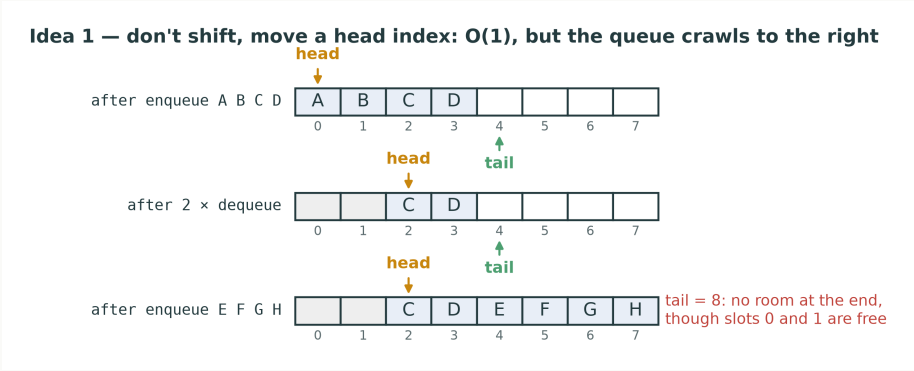
The Obvious Design

SlowQueue: a DynamicArray, as it is



- enqueue = append at the end: amortised $O(1)$.
- dequeue = `pop(0)`: every remaining element shifts left — $O(n)$.

Idea 1: don’t shift — move the front

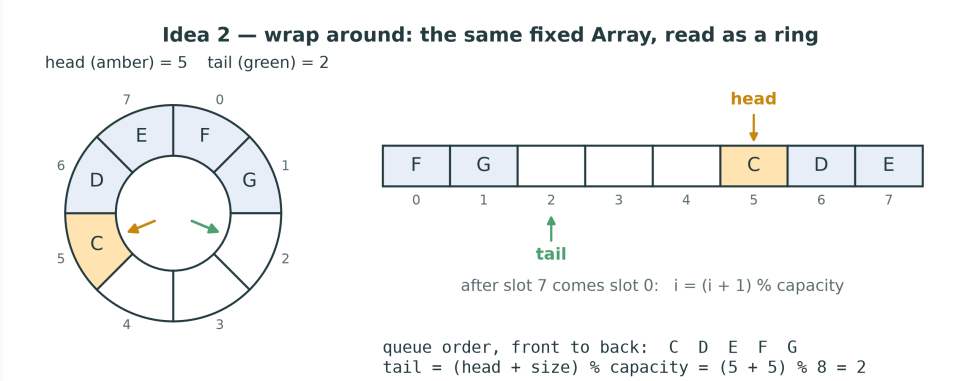


Keep a head index. Dequeue reads `block[head]` and adds 1 to head: $O(1)$.

But the used part crawls right, and the slots it leaves behind are never used again.

The Ring Buffer

Idea 2: wrap around



After the last slot comes slot 0 again:

- **front** at head; **back** at `(head + size) % capacity`
- every step forward is `(i + 1) % capacity`

The two operations

enqueue(x)

1. full? — see the next slides
2. write x at `(head + size) % capacity`
3. `size += 1`

dequeue()

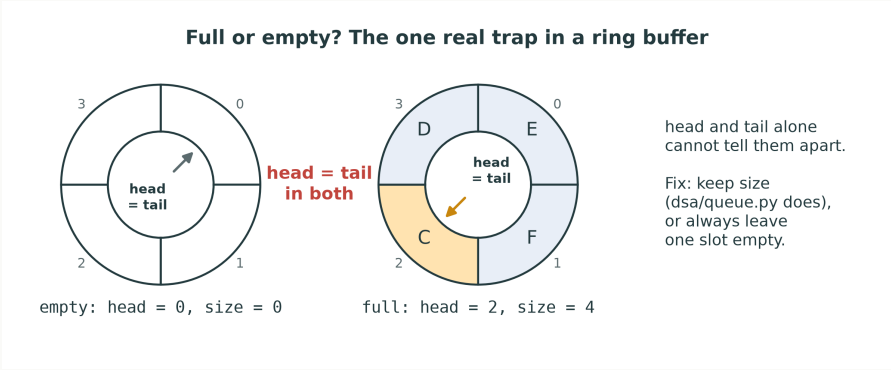
1. empty? — `IndexError`
2. read the value at `head`; clear that slot
3. `head = (head + 1) % capacity`; `size -= 1`

Nothing moves. Every step is a little arithmetic: $O(1)$.

Trace: capacity 4

Operation	Block	head	size	Returns
start	_ _ _ _	0	0	
enqueue A, B, C	A B C _	0	3	
dequeue	_ B C _	1	2	A
dequeue	_ _ C _	2	1	B
enqueue D	_ _ C D	2	2	
enqueue E	E _ C D	2	3	← wrapped to slot 0
enqueue F	E F C D	2	4	← full
dequeue	E F _ D	3	3	C

Full or empty?



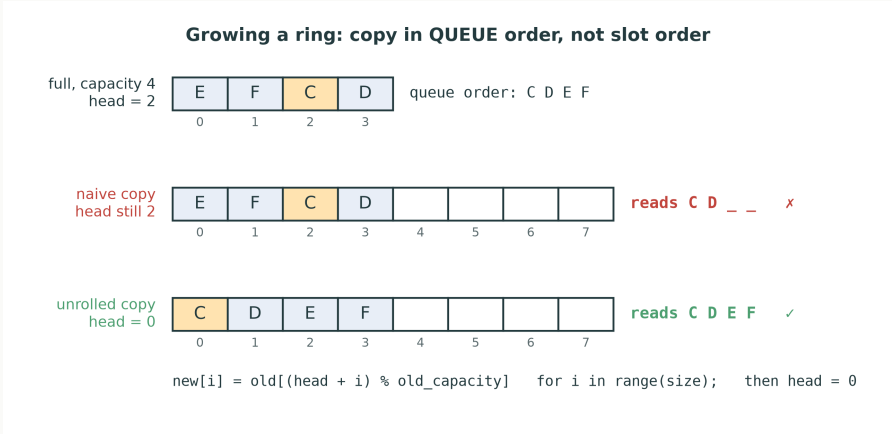
When `head == tail`, the ring is **either** empty **or** full. Head and tail alone cannot tell which.

When the ring is full

Two honest choices:

- **Refuse:** raise an error. A *bounded* queue — fixed memory, like a real buffer.
- **Grow** (the challenge): allocate a bigger `Array` and copy — as `DynamicArray` does, doubling so that enqueue stays amortised $O(1)$.

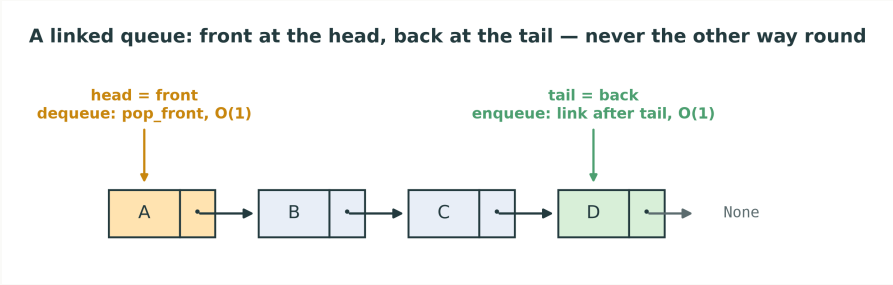
Growing: the trap



Copying slot by slot keeps the wrapped order — so the queue reads **wrong**. Copy in **queue order**, and restart head at 0.

Two More Honest Queues

The linked queue



Front at the **head**, back at the **tail**: both operations $O(1)$ worst case.

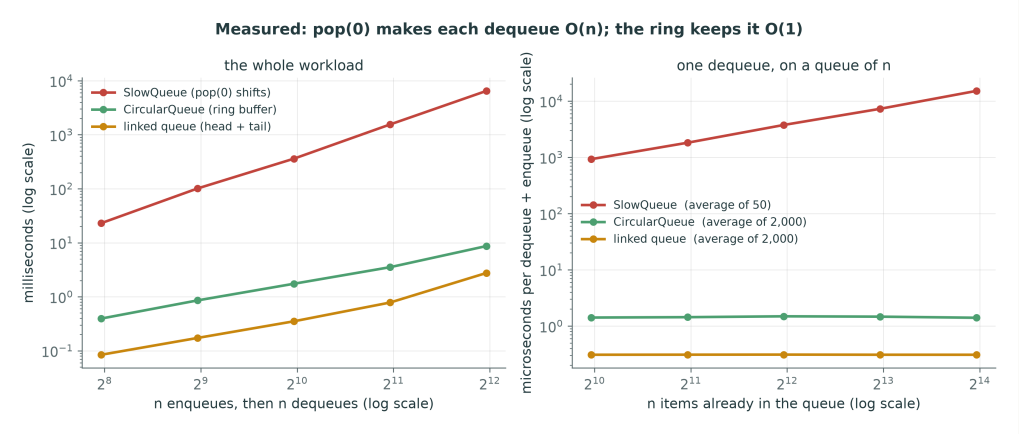
A queue from two stacks

- `inbox`: enqueue pushes here.
- `outbox`: dequeue pops here. When `outbox` is **empty**, first pour **all** of `inbox` into it.

Pouring reverses the order — and LIFO reversed is FIFO.

Measured

The gap is the lesson



Five designs, compared

	enqueue	dequeue	memory
`SlowQueue` — array, `pop(0)`	amortised $O(1)$	$O(n)$	compact
Array, head index, no wrap	amortised $O(1)$	$O(1)$	**dead slots** grow for ever
`CircularQueue` — ring buffer	$O(1)$ (amortised if it grows)	$O(1)$	compact, reuses slots
Linked, head + tail	$O(1)$	$O(1)$	a node per item
Two stacks	amortised $O(1)$	amortised $O(1)$	two arrays

Where Queues Are Used

FIFO is fairness — and breadth

- **Scheduling:** print jobs, requests to a web server, processes sharing a CPU (round-robin).
- **Buffers:** keyboard input, network packets, audio — ring buffers between a producer and a consumer; "the last k values" for a moving average.
- **Breadth-first search** (Week 14): visit everything 1 step away, then 2 steps, … — the queue keeps the frontier in order.
- **Level-order traversal** of a tree (Week 11): the same idea on a tree.

The deque: both ends, both ways

A **deque** (double-ended queue, say “deck”): add and remove at **both** ends, all $O(1)$.

- only one end used → a **stack**
- add at one end, remove at the other → a **queue**

In Python: `collections.deque` — `append`, `appendleft`, `pop`, `popleft`.

Watch it run: the Queue Lab

Queue Lab — open it in any browser.

- The code, the variables and the block drawn as a **strip and as a ring**
- `SlowQueue` shifting on every dequeue; the head index that **crawls**
- The ring buffer: this lecture's trace, a **wrap**, a ring that **grows**
- Full or empty; the linked queue; a queue from **two stacks**
- Josephus, binary numbers, a moving average, a deque on a ring
- *What happens next?* — answer before you step
- Ten scenarios that **go wrong**: no `%`, a naive grow, a stale `tail`…

This Week

Exercises: `dsa/queue.py`

Method	Target	The trap
`SlowQueue.enqueue`	amortised $O(1)$	`append` on `self._items`
`SlowQueue.dequeue`	$O(n)$ — on purpose	`IndexError` when empty; `pop(0)`
`CircularQueue.enqueue`	$O(1)$	tail = `(head + size) % capacity`; full?
`CircularQueue.dequeue`	$O(1)$	`IndexError` when empty; head wraps
challenge: grow when full	amortised $O(1)$	copy in queue order; head = 0

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

Homework 7 — before Lecture 08

1. **Implement** `dsa/queue.py` until the queue tests pass — then the challenge: grow instead of raising, and test it on a ring that has wrapped.
2. **Measure** `SlowQueue` against `CircularQueue` in `notebooks/07-queues.ipynb` and plot both. Estimate how long the `SlowQueue` would take for n = 10^6.
3. **Trace** a `CircularQueue(capacity=3)` through: enqueue 1, 2, 3; dequeue; enqueue 4; dequeue; dequeue; enqueue 5, 6. Show the block, head and size after each step.
4. **Design.** A queue from two stacks: write it, and explain why it is amortised $O(1)$ (W7-C1).

Summary

Seven things to keep

1. A queue is **FIFO**: enqueue at the **back**, dequeue at the **front**, both $O(1)$.
2. Empty `dequeue` raises **`IndexError`**.
3. On a plain array, one end is always the front: `pop(0)` or `insert(0)` — $O(n)$.
4. The **ring buffer**: `head` moves, nothing shifts; tail = `(head + size) % capacity`.
5. `head == tail` means **full or empty**: keep `size` (or waste one slot).
6. Growing a ring: copy in **queue order**, then `head = 0`.
7. Linked queue: front at the **head**, back at the **tail**. Two stacks: **amortised** $O(1)$.

Next

Week 8 — Searching *(midterm window)*. Seven ways to find a value, and what a **sorted** array buys you: from $O(n)$ to $O(\log n)$.